อ่านเตรียมตอบอาจารย์

ระบบจัดการคำสั่งซื้อและตรวจสอบคลังสินค้า ร้านวัสดุก่อสร้างอำพรคอนกรีต (SOML)

เอกสารอ่านเองเพื่อทำความเข้าใจ ไม่ใช่เอกสารส่ง · ข้อมูล ณ วันที่ 27/09/2569

สิ่งที่อาจารย์ขอ	อ่านที่หัวข้อ
ใน use case ไปครอบคลุม requirement ข้อไหนบ้าง	8 และ 12
ทุก use case มี step ทำงาน โปรแกรมต้องทำงานตามนั้น	9 (มีเลขบรรทัดในโค้ด และบอกว่าเปิดบทไหนดู)
ทุก use case มี acceptance criteria เป็นเงื่อนไขยอมรับ	10
ทุก use case มี UI และ data ที่ใช้ ซึ่งนำไปสู่ ERD	9 และ 11
มองออกไหมว่าอะไรสัมพันธ์กับอะไร	1 และ 12

ข้อควรระวัง: ตัวอย่างสถานการณ์ในตาราง EV และตัวอย่างในตาราง FR เขียนขึ้นเพื่อให้เข้าใจง่าย ไม่ใช่ตัวเลขที่เก็บได้จริงจากร้าน ห้ามพูดกับอาจารย์ว่าเป็นข้อมูลจริง ส่วนตัวเลขที่เป็นผลจริงคือคอลัมน์ “ผลจริง” และ “ผล” ซึ่งมาจากบทที่ 4

1. ภาพรวม: อะไรสัมพันธ์กับอะไร

ทุกอย่างในเล่มต่อกันเป็นสายเดียว เริ่มจากปัญหาจริงที่ร้าน แล้วลงไปจนถึงโค้ดและตารางในฐานข้อมูล ถ้าจำสายนี้ได้ จะตอบคำถามของอาจารย์ได้ทุกข้อ

ลำดับ	สิ่งนั้น	คำถามที่มันตอบ	ตัวอย่างสายเดียวกัน (เรื่องจ่ายของซ้ำ)
1	EV หลักฐานจากหน้างาน	ร้านมีปัญหาอะไรจริง	EV-06 มีคนเอาบิลเก่ามารับของซ้ำ
2	วัตถุประสงค์ (บทที่ 1)	โครงงานจะแก้อะไร	1.2.3 ป้องกันการรับสินค้าซ้ำ
3	FR / NFR / BR	ระบบต้องทำอะไร ทำได้ดีแค่ไหน และต้องเคารพกติกาอะไร	FR-06 ตัดสต็อกในธุรกรรมเดียวและกันจ่ายซ้ำ · NFR-04 ทนธุรกรรมพร้อมกัน · BR-02 สต็อกต้องตรงของจริง
4	US เรื่องเล่าของผู้ใช้	ผู้ใช้อยากได้อะไร พูดด้วยภาษาผู้ใช้	US-03 พนักงานคลังอยากให้ระบบกันจ่ายของซ้ำ จะได้ไม่ต้องจำเอง
5	UC กรณีการใช้งาน	ใครทำอะไรกับระบบ ทีละขั้น	UC-07 ยืนยันจ่ายสินค้าและตัดสต็อก
6	ขั้นตอนใน UC	โปรแกรมต้องทำอะไรตามลำดับ	ขั้น 4 เปลี่ยนสถานะแบบมีเงื่อนไข ถ้าถูกจ่ายไปแล้วจะไม่พบแถว
7	AC เกณฑ์การยอมรับ	ทำงานแบบไหนจึงถือว่ายอมรับได้	AC-07 ส่งคำขอจ่ายพร้อมกันสองคำขอ สำเร็จแค่หนึ่ง สต็อกหักครั้งเดียว
8	TC กรณีทดสอบ	ทดสอบ AC อย่างไร ผลจริงเป็นอย่างไร	TC-06 ยิงพร้อมกัน ผ่าน
9	หน้าจอ (UI)	ผู้ใช้ทำขั้นตอนนั้นที่หน้าจอไหน	หน้ายืนยันจ่ายสินค้า /dispatch/:id
10	ข้อมูล (ERD)	ขั้นตอนนั้นอ่าน/เขียนตารางไหน	orders order_items products stock_movements audit_logs
11	โค้ด (C3 โมดูล)	ส่วนไหนของโปรแกรมทำขั้นตอนนั้น	Stock Deduction Service backend/modules/stock-deduction

ประโยคสั้นที่ควรพูดได้เอง: “ปัญหาจริง (EV) ทำให้เกิดความต้องการ (FR) ความต้องการถูกเขียนเป็นเรื่องของผู้ใช้ (US) แล้วแตกเป็นกรณีการใช้งาน (UC) ที่มีขั้นตอน ขั้นตอนบอกว่าโปรแกรมต้องทำอะไร เกณฑ์การยอมรับ (AC) บอกว่าทำแบบไหนจึงผ่าน และกรณีทดสอบ (TC) คือการลองจริงตาม AC ทุก UC ใช้หน้าจอหนึ่งหน้าและอ่านเขียนตารางในฐานข้อมูล ซึ่งรวมกันเป็น ERD”

2. คำย่อทั้งหมด

คำย่อ	ชื่อเต็ม	คืออะไร	ทำไมต้องแยกเป็นรหัส
EV	Evidence	ปัญหาที่เห็นจริงที่ร้าน	เพื่อพิสูจน์ว่าความต้องการทุกข้อมีที่มา ไม่ได้คิดขึ้นเอง
FR	Functional Requirement	สิ่งที่ระบบต้องทำได้	ทดสอบได้ว่าทำได้หรือไม่
NFR	Non-functional Requirement	ระบบต้องทำได้ดีแค่ไหน (เร็ว ปลอดภัย ใช้บนมือถือได้)	ทุกข้อมีตัวเลขหรือวิธีวัด จึงตัดสินได้ว่าผ่านหรือไม่
BR	Business Rule	กติกาของร้านที่เป็นจริงอยู่แล้ว แม้ไม่มีคอมพิวเตอร์	แยกจาก FR เพราะ BR คือสิ่งที่ระบบต้องเคารพ ส่วน FR คือสิ่งที่ระบบทำเพื่อให้ BR เป็นจริง
US	User Story	ความต้องการในภาษาผู้ใช้ “ในฐานะ… ฉันต้องการ… เพื่อ…”	ให้เจ้าของร้านอ่านแล้วยืนยันได้ว่าตรงใจก่อนออกแบบ
UC	Use Case	ใคร (ผู้กระทำ) ทำอะไรกับระบบ ทีละขั้น	ใช้ตัดสินว่าโปรแกรมมีโมดูลอะไร และทำงานตามลำดับไหน
AC	Acceptance Criteria	เงื่อนไขตั้งต้น → การกระทำ → ผลที่ต้องเกิด	ใช้ตัดสินว่ายอมรับได้หรือไม่ แปลงเป็นกรณีทดสอบได้ตรง ๆ
TC	Test Case	วิธีทดสอบจริงพร้อมผลที่คาดหวัง	เป็นหลักฐานว่า AC ผ่านจริง
ERD	Entity Relationship Diagram	แผนภาพตารางในฐานข้อมูลและความสัมพันธ์	บอกว่าข้อมูลที่ทุก UC ใช้เก็บอยู่ที่ไหน
C1 C2 C3	C4 Model ระดับ 1–3	ภาพระบบกับโลกภายนอก · ส่วนที่ติดตั้งแยกกัน · โมดูลภายในหลังบ้าน	ดูการออกแบบจากกว้างไปแคบ

เรื่อง BRD: เล่มของเราไม่มีเอกสาร BRD (Business Requirements Document) แยกเป็นเล่ม สิ่งที่ทำหน้าที่เดียวกันในเล่มคือ ปัญหาในบทที่ 1 หัวข้อ 1.1 + EV (ตารางที่ 3.2) + วัตถุประสงค์ 1.2 + BR (ตารางที่ 3.5) ถ้าอาจารย์ถามถึง BRD ให้ตอบตามนี้ อย่าบอกว่ามีเอกสาร BRD

3. หลักฐานจากหน้างาน (EV) และวัตถุประสงค์

ได้มาจากการไปดูการทำงานจริงที่ร้านและถามเจ้าของร้านกับพนักงาน ดูในบทที่ 3 ตารางที่ 3.2

วัตถุประสงค์	ใจความ
1.2.1	ลดข้อผิดพลาดจากการใช้กระดาษในการบันทึกการขายและจัดการสต็อก
1.2.2	ลดเวลารอของลูกค้า ให้คลังเริ่มจัดของได้ทันทีที่ชำระเงิน
1.2.3	สต็อกในระบบต้องตรงของจริง และกันการรับของซ้ำ
1.2.4	ผู้บริหารเห็นยอดขาย สต็อก คิวงานแบบเรียลไทม์ และสั่งของจากข้อมูลจริง

EV	ปัญหาที่เห็น	ตัวอย่างสถานการณ์	ทำไมถือเป็นปัญหา	วัตถุประสงค์
EV-01	ไม่มีระบบบันทึกการขายที่เชื่อมกับสต็อก ใช้บิลกระดาษ	ขายเหล็ก 10 เส้น แต่ไม่มีที่ไหนหักยอดเหล็กในคลัง	ขายไปเท่าไรไม่รู้ สต็อกไม่ขยับตาม	1.2.1
EV-02	ข้อมูลการขายกระจัดกระจาย ตรวจย้อนหลังยาก	ยอดเงินไม่ตรง แต่หาไม่ได้ว่าบิลไหนใครขาย	ตรวจสอบความผิดพลาดไม่ได้	1.2.1
EV-03	หน้าร้านกับคลังอยู่คนละที่ ไม่มีช่องทางสื่อสารทันที	คลังไม่รู้ว่ามีคนซื้อจนลูกค้าเดินมาถึง	คลังเริ่มงานช้า	1.2.2
EV-04	ลูกค้าถือใบเสร็จเดินไปคลังเอง เกิดคอขวด	ช่วงเที่ยงลูกค้าต่อแถวที่คลัง	ลูกค้ารอนาน	1.2.2
EV-05	คลังอ่านใบเสร็จด้วยตา หยิบผิดประเภทผิดจำนวน	ลายมืออ่านยาก 4 หุนกับ 5 หุน สับสน	ส่งของผิด ต้องแก้ทีหลัง	1.2.3
EV-06	มีคนเอาบิลเก่ามารับของซ้ำ	บิลเดิมถูกใช้รับของสองครั้ง	ร้านเสียของฟรี	1.2.3
EV-07	สต็อกในบันทึกไม่ตรงของจริง	บันทึกเหลือ 50 ถุง นับจริงเหลือ 42	สั่งของผิดจังหวะ	1.2.3
EV-08	ผู้จัดการต้องรอสรุปยอดและนับสต็อกตอนท้ายวัน	รู้ยอดขายวันนี้ตอนปิดร้านเท่านั้น	ตัดสินใจระหว่างวันไม่ได้	1.2.4
EV-09	สั่งซื้อสินค้าจากความรู้สึก ไม่ใช่ข้อมูล	ของหมดแล้วจึงรู้ว่าต้องสั่ง	ของขาดมือ ขายไม่ได้	1.2.4

เหตุผลที่ต้องมี EV: ทุกความต้องการต้องย้อนกลับไปหาปัญหาจริงได้ ถ้าอาจารย์ชี้ FR ข้อใดแล้วถามว่า “ข้อนี้มาจากไหน” ต้องตอบได้ว่ามาจาก EV ข้อไหน

4. ความต้องการเชิงหน้าที่ (FR)

ดูในบทที่ 3 ตารางที่ 3.3

FR	ระบบต้องทำ	มาจาก	เหตุผลที่ต้องมี	ตัวอย่างที่เห็นได้ในระบบ
FR-01	เข้าสู่ระบบและควบคุมสิทธิ์ตามบทบาท	EV-02	ต้องรู้ว่าใครทำรายการ ประวัติจึงมีความหมาย และพนักงานขายต้องแก้ราคาสินค้าไม่ได้	พนักงานขายเห็นแค่เมนูขายหน้าร้าน ถ้าพยายามเรียกหน้าผู้จัดการเซิร์ฟเวอร์ปฏิเสธ
FR-02	จัดการสินค้า หมวดหมู่ ราคา หน่วย จุดสั่งซื้อเพิ่ม	EV-09	จุดสั่งซื้อเพิ่มคือเกณฑ์ที่ใช้เตือนของใกล้หมด ถ้าไม่มีข้อนี้ FR-08 ทำงานไม่ได้	ตั้งเหล็ก 5 หุน จุดสั่งซื้อเพิ่ม 20 เส้น
FR-03	สร้างคำสั่งซื้อ คำนวณยอด แสดง QR พิมพ์ใบเสร็จ	EV-01	แทนบิลกระดาษ ยอดคำนวณจากราคาในระบบ ไม่ใช่คนคิดเอง	เลือกเหล็ก 2 เส้น × 65 บาท ระบบคิด 130 บาท แสดง QR 130 บาท
FR-04	คิวรอจ่ายอัปเดตเองบนหน้าจอคลัง	EV-03, EV-04	คลังต้องรู้ก่อนลูกค้าเดินไปถึง	กดยืนยันรับเงินที่หน้าร้าน ไม่ถึงวินาทีการ์ดใหม่ขึ้นที่คลัง
FR-05	คลังยืนยันจ่ายสินค้าจากรายการในคิว	EV-05	คลังดูรายการจากจอ ไม่ต้องอ่านลายมือ	หน้าจอแสดง “2 เส้น” ตัวใหญ่พร้อมยอดในคลัง
FR-06	ตัดสต็อกในธุรกรรมเดียวกับเปลี่ยนสถานะ และกันจ่ายซ้ำ	EV-06, EV-07	ถ้าเปลี่ยนสถานะสำเร็จแต่หักสต็อกพลาด ยอดจะเพี้ยน ต้องสำเร็จพร้อมกันหรือไม่สำเร็จเลย	สองเครื่องกดจ่ายใบเดียวกันพร้อมกัน สำเร็จแค่เครื่องเดียว
FR-07	แดชบอร์ดยอดขาย สต็อก คิวงาน	EV-08	ผู้จัดการไม่ต้องรอสรุปตอนปิดร้าน	ยอดขายวันนี้ 12,480 บาท คิวค้าง 4 ใบ
FR-08	แจ้งเตือนในแอปเมื่อสินค้าต่ำกว่าจุดสั่งซื้อเพิ่มหรือคิวค้าง	EV-09	เตือนก่อนของหมด ไม่ต้องรอให้คนเปิดดู	จ่ายเหล็กจนเหลือ 45 เส้น ต่ำกว่า 50 ระบบสร้างการแจ้งเตือนเอง
FR-09	บันทึกประวัติเหตุการณ์สำคัญและค้นย้อนหลัง	EV-02	ตอบได้ว่าใครทำอะไรเมื่อไร	ค้นได้ว่าสันติจ่าย ORD-…-0015 เวลา 14:31 จากเครื่องไหน

5. ความต้องการนอกเหนือหน้าที่ (NFR)

ดูในบทที่ 3 ตารางที่ 3.4 ทุกข้อต้องมีวิธีวัด มิฉะนั้นตัดสินไม่ได้ว่าผ่าน

NFR	ต้องทำได้	ตัวเลขนี้มาจากไหน / เหตุผล	ผลจริง (บทที่ 4)
NFR-01	ทุกคำขอตรวจโทเคนและสิทธิ์ที่เซิร์ฟเวอร์ รหัสผ่านเก็บแบบเข้ารหัส	ซ่อนเมนูอย่างเดียวไม่พอ คนรู้ที่อยู่ของเส้นทางก็เรียกตรงได้	ผ่าน
NFR-02	คำสั่งซื้อใหม่ขึ้นหน้าคิวภายใน 3 วินาที	ลูกค้าเดินจากจุดจ่ายเงินไปคลังราวหนึ่งนาที ไม่กี่วินาทีจึงพอให้คลังเริ่มก่อน	ผ่าน สูงสุด 80 มิลลิวินาที
NFR-03	หน้าคิวและหน้าจ่ายใช้บนแท็บเล็ตและมือถือได้	พนักงานคลังเดินอยู่ในโกดัง ไม่ได้นั่งหน้าคอม	ผ่าน ตรวจที่กว้าง 390 พิกเซล
NFR-04	ทนธุรกรรมพร้อมกันโดยไม่เกิดสภาวะแข่งขัน	ตอนเร่งด่วนมีหลายเครื่องทำงานพร้อมกัน	ผ่าน
NFR-05	พิมพ์ใบเสร็จเสร็จภายใน 5 วินาที	ลูกค้าไม่ควรยืนรอใบเสร็จนาน	ยังไม่ทวนสอบ ต้องต่อเครื่องพิมพ์จริงที่ร้าน
NFR-06	รองรับ 10 เซสชันพร้อมกัน	จำนวนพนักงานที่ทำงานพร้อมกันจริงในร้าน	ผ่าน ไม่มีคำขอล้มเหลว
NFR-07	ทำงานบน Chrome Edge Firefox รุ่นล่าสุด	เครื่องในร้านใช้เบราว์เซอร์ทั่วไป	ผ่านบางส่วน ทดสอบแล้วบนเบราว์เซอร์ที่ใช้รันชุดทดสอบ
NFR-08	แยกชั้นและแยกส่วนตามที่ออกแบบ	แก้ส่วนหนึ่งไม่กระทบส่วนอื่น	ผ่าน โครงโฟลเดอร์ตรงกับ C3
NFR-09	ติดตั้งซ้ำได้ด้วยเอกสารอย่างเดียว	ร้านต้องติดตั้งเองได้ถ้าเครื่องเสีย	ยังไม่ทวนสอบ

6. กฎทางธุรกิจ (BR)

ดูในบทที่ 3 ตารางที่ 3.5 BR คือกติกาของร้าน ระบบต้องเคารพ FR คือสิ่งที่ระบบทำเพื่อให้กติกานั้นเป็นจริง

BR	กติกาของร้าน	บังคับการออกแบบอย่างไร	ถ้าไม่ทำตามจะเกิดอะไร
BR-01	ออกใบเสร็จกับส่งเข้าคิวคลังต้องเกิดจากการกระทำเดียวกัน	ปุ่ม “ยืนยันรับชำระเงิน” เป็นจุดเดียวที่ทั้งบันทึก พิมพ์ และส่งเข้าคิว	ลูกค้าได้ใบเสร็จแต่คลังไม่รู้ หรือคลังจัดของแต่ไม่มีใบเสร็จ
BR-02	สต็อกในระบบต้องตรงของจริง	จ่ายของกับหักสต็อกอยู่ในธุรกรรมเดียว และเฉพาะคลังเป็นคนกดจ่าย	ยอดเพี้ยนเหมือนเดิม
BR-03	ต้องลดเวลารอของลูกค้า	เซิร์ฟเวอร์ผลักข้อมูลไปหน้าจอคลังเอง (WebSocket) ไม่รอหน้าจอมาถาม	คลังรู้ช้า ลูกค้ารอ
BR-04	ผู้บริหารต้องเห็นยอดและคิวได้ตลอดเวลา	ตัวเลขคำนวณสดจากฐานข้อมูลทุกครั้ง ไม่เก็บค่าสรุป	ตัวเลขบนจอค้าง ไม่ตรงความจริง
BR-05	ต้องแจ้งเตือนเชิงรุกเพื่อลดของขาดมือ	ระบบตรวจเองหลังสต็อกเปลี่ยนทุกครั้ง	ต้องรอคนมาเปิดดูจึงรู้

7. ผู้ใช้และเรื่องเล่าของผู้ใช้ (US)

บทบาท	งาน	หน้าแรกหลังเข้าระบบ
พนักงานขาย	สร้างคำสั่งซื้อ แสดง QR ยืนยันรับเงิน พิมพ์ใบเสร็จ	ขายหน้าร้าน
พนักงานคลังสินค้า	ดูคิว ปรับสถานะการจัดของ ยืนยันจ่ายสินค้า	คิวรอจ่ายสินค้า
ผู้จัดการ	จัดการสินค้า ดูแดชบอร์ด ตรวจประวัติ รับการแจ้งเตือน	แดชบอร์ดปฏิบัติการ
ระบบอัตโนมัติ	ออกเลขที่คำสั่งซื้อ ตรวจเงื่อนไขและสร้างการแจ้งเตือน	ไม่มี (ไม่ใช่คน)

เหตุผลที่ไม่มีบทบาทผู้ดูแลระบบ: ร้านเล็ก ไม่มีคนดูแลระบบเต็มเวลา งานจัดการผู้ใช้จึงรวมไว้ที่ผู้จัดการ

US	ในฐานะ	ฉันต้องการ	เพื่อที่จะ	กลายเป็น UC	FR
US-01	พนักงานขาย	บันทึกคำสั่งซื้อและออกใบเสร็จในขั้นตอนเดียว	ไม่ต้องเขียนบิลและคิดยอดเอง	UC-01, UC-03, UC-04	FR-01, FR-03
US-02	พนักงานคลัง	เห็นคำสั่งซื้อใหม่เองโดยไม่กดรีเฟรช	เริ่มจัดของก่อนลูกค้ามาถึง	UC-05, UC-06	FR-04
US-03	พนักงานคลัง	ให้ระบบกันจ่ายของซ้ำจากใบเดิม	ไม่ต้องจำเองว่าใบไหนจ่ายแล้ว	UC-07	FR-05, FR-06
US-04	ผู้จัดการ	เห็นยอดขายและคิวระหว่างวันทันที	ไม่ต้องรอตอนปิดร้าน	UC-08, UC-09	FR-07
US-05	ผู้จัดการ	ให้ระบบเตือนเมื่อของใกล้หมด	สั่งของได้ทัน	UC-02, UC-11	FR-02, FR-08
US-06	ผู้จัดการ	ค้นย้อนหลังได้ว่าใครทำอะไรเมื่อไร	ตรวจสอบได้เมื่อยอดไม่ตรง	UC-10	FR-09

ทำไมต้องมี US ก่อน UC: US เขียนด้วยภาษาผู้ใช้ เจ้าของร้านอ่านแล้วบอกได้ว่าใช่หรือไม่ใช่ ส่วน UC เป็นรายละเอียดทางเทคนิคที่เจ้าของร้านอ่านไม่ออก

8. กรณีการใช้งานทั้ง 11 กรณี และความต้องการที่ครอบคลุม

ดูในบทที่ 3 ตารางที่ 3.8 คำตอบข้อแรกของอาจารย์ “ใน use case ไปครอบคลุม requirement ข้อไหนบ้าง” อยู่ในตารางนี้

UC	ชื่อ	ผู้กระทำหลัก	ครอบคลุม	ทำไมต้องมี UC นี้
UC-01	เข้าสู่ระบบและควบคุมสิทธิ์	ทุกบทบาท	FR-01, NFR-01	ทุกงานต้องผ่านการยืนยันตัวตนก่อน
UC-02	จัดการสินค้าและจุดสั่งซื้อเพิ่ม	ผู้จัดการ	FR-02	ไม่มีข้อมูลสินค้าก็ขายไม่ได้ ไม่มีจุดสั่งซื้อเพิ่มก็เตือนไม่ได้
UC-03	สร้างคำสั่งซื้อและออกใบเสร็จ	พนักงานขาย	FR-03, BR-01, NFR-05	จุดเริ่มต้นของทุกคำสั่งซื้อ
UC-04	สร้างเลขที่คำสั่งซื้อไม่ซ้ำ	ระบบอัตโนมัติ	FR-03	แยกออกมาเพราะมีเงื่อนไขของตัวเองที่ต้องทดสอบแยก (สองเครื่องขอเลขพร้อมกัน)
UC-05	ดูคิวรอจ่ายแบบเรียลไทม์	พนักงานคลัง	FR-04, NFR-02, BR-03	คลังรู้ก่อนลูกค้ามาถึง
UC-06	ปรับสถานะการจัดของ	พนักงานคลัง	FR-04	บอกคนในคลังว่ามีคนรับใบนี้ไปจัดแล้ว ไม่จัดซ้ำกัน
UC-07	ยืนยันจ่ายสินค้าและตัดสต็อก	พนักงานคลัง	FR-05, FR-06, NFR-04, BR-02	หัวใจของระบบ แก้ปัญหาจ่ายซ้ำและสต็อกเพี้ยน
UC-08	แดชบอร์ดปฏิบัติการ	ผู้จัดการ, คลัง	FR-07, BR-04	ดูสถานการณ์ระหว่างวัน
UC-09	แดชบอร์ดระดับบริหาร	ผู้จัดการ	FR-07	ดูแนวโน้มย้อนหลังเพื่อตัดสินใจสั่งของ
UC-10	ค้นหาประวัติการทำรายการ	ผู้จัดการ	FR-09	ตรวจสอบย้อนหลังเมื่อยอดไม่ตรง
UC-11	รับการแจ้งเตือนในแอป	ระบบ, ผู้จัดการ, คลัง	FR-08, BR-05	เตือนก่อนของหมดหรือคิวค้าง

ตรวจกลับ: FR ทุกข้อ FR-01 ถึง FR-09 มี UC รองรับอย่างน้อยหนึ่งกรณี ไม่มี FR ที่ลอย และไม่มี UC ที่ไม่มี FR (ดูหัวข้อ 12)

9. ทีละ UC: ขั้นตอน → โปรแกรม (บรรทัดไหน) ⟶ หน้าจอ → ข้อมูล → AC

คำพูดของอาจารย์ “ทุก use case มี step ทำงาน โปรแกรมต้องทำงานตามนั้น” หมายความว่าทุกขั้นตอนใน UC ต้องมีโค้ดที่ทำจริง หัวข้อนี้บอกทั้งไฟล์และบรรทัดในโค้ด และบอกว่าต้องเปิดเล่มบทไหน หัวข้อไหน ตารางหรือรูปที่เท่าไร ดูไปพร้อมกัน

วิธีอ่านตำแหน่งในโค้ด: ทุกไฟล์อยู่ในโฟลเดอร์ soml/ เช่น backend/modules/auth/index.js : 39 คือเปิดไฟล์ soml/backend/modules/auth/index.js แล้วดูบรรทัดที่ 39 ถ้าเปิดใน VS Code กด Ctrl+G (ใช้ได้ทั้ง Windows และ Mac) แล้วพิมพ์เลขบรรทัด เลขบรรทัดตรงกับโค้ด ณ วันที่ 27/09/2569 ถ้าแก้โค้ดภายหลัง บรรทัดอาจเลื่อน เส้นทางบริการทุกเส้นประกาศรวมไว้ที่ backend/app.js พร้อมสิทธิ์ของแต่ละเส้น

ตำแหน่งในเล่มที่ใช้กับทุก UC

อยากดู	เปิดที่
รายชื่อ UC ทั้งหมดและ FR ที่รองรับ	บทที่ 3 หัวข้อ 3.3.3 ตารางที่ 3.8
ขั้นตอนเต็มของแต่ละ UC	Use Case X หัวข้อ 4.1 (UC-01) ถึง 4.11 (UC-11)
AC-01 ถึง AC-11	บทที่ 3 หัวข้อ 3.3.6 ตารางที่ 3.11
AC-12 ถึง AC-15	Use Case X ภาคผนวก D
โมดูลหลังบ้าน (C3)	บทที่ 3 หัวข้อ 3.4.1 ตารางที่ 3.13 และหัวข้อ 3.6 รูปที่ 3.8
หน้าจอและบทบาท	บทที่ 3 หัวข้อ 3.4.2 ตารางที่ 3.15 และหัวข้อ 3.9.2 ตารางที่ 3.21
เส้นทางบริการ	บทที่ 3 ตารางที่ 3.18
ERD และตารางข้อมูล	บทที่ 3 หัวข้อ 3.7.1 รูปที่ 3.9 ตารางที่ 3.19
กรณีทดสอบ TC	บทที่ 3 หัวข้อ 3.10.2 ตารางที่ 3.23 · ผลจริงบทที่ 4 ตารางที่ 4.5

UC-01 เข้าสู่ระบบและควบคุมสิทธิ์ตามบทบาท

ขั้นตอน	โปรแกรมทำอะไร	ไฟล์ : บรรทัด
1 กรอกชื่อผู้ใช้และรหัสผ่าน	หน้าเข้าสู่ระบบส่งไป POST /api/auth/login	frontend/src/pages/login/index.jsx : 30–36 · backend/app.js : 32
2 เทียบรหัสผ่านกับค่าที่เข้ารหัสไว้	อ่านผู้ใช้จาก users แล้วใช้ bcrypt เทียบ	backend/modules/auth/index.js : 35, 39
3 ตรวจว่าบัญชียังเปิดใช้	ผิดเงื่อนไขใดก็ตอบข้อความเดียวกัน	backend/modules/auth/index.js : 20, 40
4 ออกโทเคนที่ระบุบทบาท และบันทึก LOGIN	jwt.sign แล้วเขียน audit_logs	backend/modules/auth/index.js : 43, 44
5 พาไปหน้าแรกของบทบาท	ขาย → /pos · คลัง → /queue · ผู้จัดการ → /dashboard	frontend/src/services/session.jsx : 13 · login/index.jsx : 36
6 ทุกคำขอตรวจโทเคนและบทบาทที่เซิร์ฟเวอร์	verifyToken แล้ว requireRole ก่อนเข้าโมดูล	backend/middleware/auth.js : 23, 42 · backend/app.js : 42–59
(หน้าจอ) เห็นเฉพาะเมนูของบทบาท	กรองเมนูตามบทบาท	frontend/src/components/Layout.jsx : 14, 30

เปิดดูในเล่ม: Use Case X หัวข้อ 4.1 · บทที่ 3 ตารางที่ 3.6 (หน้าแรกของบทบาท) รูปที่ 3.2 ตารางที่ 3.11 (AC-01) · บทที่ 4 หัวข้อ 4.1.1 รูปที่ 4.1

ข้อมูล: อ่าน users เขียน audit_logs · AC: AC-01, AC-12 · TC-01

ตัวอย่าง: ศริพรรณ (พนักงานขาย) เข้าระบบ เห็นแค่เมนูขายหน้าร้าน ถ้าส่งคำขอแก้ราคาสินค้า (PUT /api/products/:id) ตรง ๆ โดยไม่ผ่านหน้าจอ เซิร์ฟเวอร์ตอบ 403 เพราะ backend/app.js : 54 กำหนดว่าเส้นนี้เฉพาะผู้จัดการ เหตุผลที่ต้องตรวจที่เซิร์ฟเวอร์: การซ่อนเมนูเป็นแค่ความสะดวก ไม่ใช่การป้องกัน

UC-02 จัดการข้อมูลสินค้าและจุดสั่งซื้อเพิ่ม

ขั้นตอน	โปรแกรมทำอะไร	ไฟล์ : บรรทัด
1 เปิดหน้าสินค้า กรองหมวดหรือค้นหา	อ่านรายการจาก products	backend/modules/product/index.js : 53 · backend/app.js : 36
2 เพิ่มสินค้าใหม่	บันทึกสินค้าพร้อมประวัติในธุรกรรมเดียว	backend/modules/product/index.js : 84, 94 · backend/app.js : 53
3 แก้ไขราคา หมวด จุดสั่งซื้อเพิ่ม	บันทึกค่าใหม่พร้อมประวัติ	backend/modules/product/index.js : 106, 121 · backend/app.js : 54
3 ปรับยอดคงเหลือ	ล็อกแถวสินค้า (FOR UPDATE) แล้วเขียนการเคลื่อนไหวสต็อก	backend/modules/product/index.js : 143, 155, 162
4 บันทึกประวัติ แล้วตรวจการแจ้งเตือน	เขียน audit_logs ในธุรกรรม · ตรวจสินค้าใกล้หมดหลังธุรกรรม	backend/modules/product/index.js : 166, 171

เปิดดูในเล่ม: Use Case X หัวข้อ 4.2 · บทที่ 3 ตารางที่ 3.13 (โมดูล 2 Product Controller) รูปที่ 3.5 ตารางที่ 3.11 (AC-02) หัวข้อ 3.7.2 · บทที่ 4 หัวข้อ 4.1.7 รูปที่ 4.9

ข้อมูล: อ่าน **products** เขียน **products stock_movements audit_logs** · AC: AC-02 · TC-02

ตัวอย่าง: ตั้งจุดสั่งซื้อเพิ่มของเหล็ก 2 หุนเป็น 50 ค่าในฐานข้อมูลเป็น 50 และประวัติมีแถว “แก้ไขสินค้า” เหตุผลที่ต้องล็อกแถวตอนปรับสต็อก (บรรทัด 155): ถ้าคลังกำลังจ่ายสินค้าตัวเดียวกันพร้อมกัน ตัวเลขจะทับกันจนเพี้ยน

UC-03 สร้างคำสั่งซื้อและออกใบเสร็จ

ขั้นตอน	โปรแกรมทำอะไร	ไฟล์ : บรรทัด
1 เลือกหมวดหรือค้นหา แตะสินค้า ใส่จำนวน	หน้าขายหน้าร้าน	frontend/src/pages/pos/index.jsx
2 คำนวณยอดจากราคาในฐานข้อมูล	หน้าจอส่งแค่รหัสสินค้ากับจำนวน หลังบ้านดึงราคาเอง	pos/index.jsx : 41 · backend/modules/order/index.js : 56–58, 92
3 แสดง QR พร้อมเพย์ตามยอด	สร้างข้อมูล QR จากยอดที่คำนวณ	backend/modules/order/index.js : 100 · pos/index.jsx : 116, 176
5 กด “ยืนยันรับชำระเงิน”	ส่ง POST /api/orders	pos/index.jsx : 65–68 · backend/app.js : 43
6 ออกเลขที่ (UC-04) บันทึกใบรายการ ประวัติ ในธุรกรรมเดียว	INSERT orders order_items และประวัติ	backend/modules/order/index.js : 114–129
7 สั่งพิมพ์ใบเสร็จ หลังบันทึกสำเร็จ	ส่งไปบริการพิมพ์ ถ้าพิมพ์ไม่ได้การขายยังสำเร็จ	backend/modules/order/index.js : 141 · pos/index.jsx : 78–80
7 ผลักเข้าคิวคลัง	ผลักเหตุการณ์ order.created	backend/modules/order/index.js : 148

เปิดดูในเล่ม: Use Case X หัวข้อ 4.3 · บทที่ 3 หัวข้อ 3.3.5 ตารางที่ 3.9 (คำอธิบาย UC-03) รูปที่ 3.3 หัวข้อ 3.8.1 รูปที่ 3.10 (แผนภาพลำดับ) รูปที่ 3.14–3.15 · บทที่ 4 หัวข้อ 4.1.2 รูปที่ 4.2–4.3

ข้อมูล: อ่าน products เขียน orders order_items audit_logs · AC: AC-03, AC-05, AC-14 · TC-03, TC-12

ตัวอย่าง: เหล็ก 2 หุน 2 เส้น × 65 + เหล็ก 5 หุน 1 เส้น × 395 = 525 บาท QR ระบุ 525 บาท ถ้าเครื่องพิมพ์ดับ ใบยังเข้าคิว และหน้าจอขึ้นข้อความตามบรรทัด 80 ของ pos/index.jsx เหตุผล: เงินรับไปแล้ว การขายเกิดขึ้นจริง จะยกเลิกเพราะกระดาษหมดไม่ได้ (BR-01)

UC-04 สร้างเลขที่คำสั่งซื้อไม่ซ้ำอัตโนมัติ

สิ่งที่เกิด	ระบบทำอะไร	ไฟล์ : บรรทัด
UC-03 ขั้น 6 ขอเลขใหม่	อ่านเลขล่าสุดของวันพร้อมล็อก แล้วบวกหนึ่ง รูปแบบ ORD-ปปปปดดวว-ลำดับ	backend/modules/order/index.js : 80–89
ใบแรกของวัน	ยังไม่มีเลขของวัน จึงเริ่มที่ 0001	backend/modules/order/index.js : 87
สองเครื่องได้เลขเดียวกัน (กรณีขอบ)	ฐานข้อมูลปฏิเสธเลขซ้ำ ระบบรอสั้น ๆ แล้วทำธุรกรรมใหม่	backend/modules/order/index.js : 33, 112, 134–136

เปิดดูในเล่ม: Use Case X หัวข้อ 4.4 · บทที่ 3 ตารางที่ 3.8 (ย่อหน้าใต้ตาราง อธิบายว่าทำไมแยก UC-04) รูปที่ 3.3 · บทที่ 4 ตารางที่ 4.5 (TC-04)

ข้อมูล: orders · AC: AC-04 · TC-04 · ผลจริง: สร้าง 120 ใบทั้งต่อเนื่องและพร้อมกัน ไม่ซ้ำเลย ไม่มีหน้าจอ เพราะผู้กระทำคือระบบ

UC-05 ดูคิวรอจ่ายสินค้าแบบเรียลไทม์

ขั้นตอน	โปรแกรมทำอะไร	ไฟล์ : บรรทัด
1 เปิดหน้าคิว โหลดคิวปัจจุบัน	ดึงใบที่รอจ่าย เรียงตามเวลาชำระ	backend/modules/queue/index.js : 87, 90 · backend/app.js : 46
2 เปิดช่องทางค้าง	WebSocket ถ้าเปิดไม่ได้ใน 3 วินาที เปลี่ยนไปใช้ SSE	frontend/src/services/events.js : 26, 28, 38
3 มีใบใหม่ เซิร์ฟเวอร์ผลัก order.created	ผลักหลังธุรกรรมยืนยันแล้วเท่านั้น	backend/modules/order/index.js : 148 · queue/index.js : 38
4–5 หน้าจอได้เหตุการณ์ แล้วโหลดคิวใหม่	ทุกเหตุการณ์ที่ขึ้นต้นด้วย order.	frontend/src/pages/queue/index.jsx : 31

เปิดดูในเล่ม: Use Case X หัวข้อ 4.5 · บทที่ 3 รูปที่ 3.4 หัวข้อ 3.5.2 (ช่องทางค้างใน C2) รูปที่ 3.16 · บทที่ 4 หัวข้อ 4.1.3 รูปที่ 4.4 หัวข้อ 4.2.5.2 (ช่องทางสำรอง)

ข้อมูล: อ่าน orders order_items products users · AC: AC-06 · TC-05

ตัวอย่าง: กดยืนยันรับเงินที่หน้าร้าน การ์ดขึ้นที่คลังใน 80 มิลลิวินาที (เกณฑ์ 3 วินาที) เหตุผลที่ให้เซิร์ฟเวอร์ผลักแทนให้หน้าจอถามทุกกี่วินาที: เร็วกว่า และไม่ต้องยิงคำขอเปล่าตลอดเวลา

UC-06 ปรับสถานะการจัดของในคิวงาน

ขั้นตอน	โปรแกรมทำอะไร	ไฟล์ : บรรทัด
1 กด “เริ่มจัดของ”	ส่ง PATCH /api/orders/:id/status	frontend/src/pages/queue/index.jsx : 36–38 · backend/app.js : 47
2 ตรวจว่าบทบาทนี้ตั้งสถานะนี้ได้	ยกเลิกได้เฉพาะผู้จัดการ	backend/modules/queue/index.js : 31, 111–112
3 ตรวจว่ามีลูกศรจากสถานะเดิมไปสถานะใหม่	รอจ่าย ⟷ กำลังจัด และไปยกเลิก	backend/modules/queue/index.js : 24, 115, 117
4 เปลี่ยนสถานะและบันทึกประวัติ ในธุรกรรมเดียว	เขียน orders audit_logs	backend/modules/queue/index.js : 121
5 ผลัก order.status ไปทุกจอ	หลังธุรกรรมยืนยันแล้ว	backend/modules/queue/index.js : 130

เปิดดูในเล่ม: Use Case X หัวข้อ 4.6 · บทที่ 3 รูปที่ 3.4 หัวข้อ 3.8.4 รูปที่ 3.13 ตารางที่ 3.20 (แผนภาพสถานะ) · บทที่ 4 หัวข้อ 4.1.3

ข้อมูล: อ่านและเขียน orders เขียน audit_logs · AC: ไม่มี (ดูหัวข้อ 13)

ตัวอย่าง: สันติกด “เริ่มจัดของ” ใบ 0016 ป้ายเปลี่ยนเป็น “กำลังจัดของ” บนทุกจอ คนอื่นจะได้ไม่หยิบซ้ำ ตาราง TRANSITIONS ที่บรรทัด 24 คือแผนภาพสถานะรูปที่ 3.13 ในรูปโค้ด

UC-07 ยืนยันจ่ายสินค้าและตัดสต็อก

ขั้นตอน	โปรแกรมทำอะไร	ไฟล์ : บรรทัด
1–2 เปิดใบจากคิว ดูรายการ จำนวน ยอดในคลัง	ดึงรายการพร้อมยอดคงเหลือ	backend/modules/order/index.js : 158, 169
3 กดยืนยัน ระบบถามซ้ำ	กล่องยืนยันอีกครั้ง	frontend/src/pages/dispatch/index.jsx : 80, 23–26
(เส้นทาง) รับคำขอแล้วเปิดธุรกรรม	Dispatch Controller ส่งงานให้ Stock Deduction	backend/app.js : 48 · backend/modules/dispatch/index.js : 16, 21
4 เปลี่ยนสถานะแบบมีเงื่อนไข	ถ้าถูกจ่ายไปแล้วจะไม่พบแถว (E1)	backend/modules/stock-deduction/index.js : 32, 36–37
5 ล็อกสินค้าเรียงตามรหัส แล้วหักเมื่อยอดพอ	ไม่พอจะยกเลิกทั้งธุรกรรม (E2)	backend/modules/stock-deduction/index.js : 42, 51, 54–55
6 บันทึกการเคลื่อนไหวสต็อกและประวัติ	เขียน stock_movements audit_logs	backend/modules/stock-deduction/index.js : 61, 69
(ปฏิเสธ) ตอบ 409 พร้อมเหตุผล	ถูกจ่ายแล้ว หรือสต็อกไม่พอ	backend/modules/dispatch/index.js : 24–25
7 ผลัก order.dispatched แล้วตรวจสินค้าใกล้หมด	หลังธุรกรรม ต่อไป UC-11	backend/modules/dispatch/index.js : 32, 33

เปิดดูในเล่ม: Use Case X หัวข้อ 4.7 · บทที่ 3 ตารางที่ 3.10 (คำอธิบาย UC-07) หัวข้อ 3.7.3 (ธุรกรรมตัดสต็อก) หัวข้อ 3.8.2 รูปที่ 3.11 (แผนภาพลำดับ) รูปที่ 3.17 ตารางที่ 3.13 (โมดูล 5 และ 6) · บทที่ 2 หัวข้อ 2.1.3 (การล็อกเชิงบวก) · บทที่ 4 หัวข้อ 4.1.4 รูปที่ 4.5–4.6 หัวข้อ 4.2.5.1

ข้อมูล: อ่านและเขียน orders order_items products · เขียน stock_movements audit_logs · AC: AC-07, AC-08 · TC-06, TC-07

ตัวอย่างกันจ่ายซ้ำ: เครื่อง A และ B กดจ่ายใบ 0018 พร้อมกัน ทั้งคู่สั่งคำสั่งที่บรรทัด 32 “เปลี่ยนเป็นส่งมอบ ถ้ายังเป็นรอจ่ายหรือกำลังจัด” เครื่องแรกที่ถึงฐานข้อมูลเปลี่ยนได้ เครื่องที่สองได้ affectedRows = 0 ที่บรรทัด 36 ระบบยกเลิกทั้งธุรกรรมและขึ้นข้อความว่าถูกจ่ายไปแล้ว

ตัวอย่างสต็อกไม่พอ: ใบสั่งเหล็ก 3 เส้นแต่เหลือ 2 คำสั่งที่บรรทัด 51 หักเฉพาะเมื่อ stock_qty >= 3 จึงไม่พบแถว บรรทัด 54 ยกเลิกทั้งหมด สถานะและสต็อกไม่เปลี่ยน

เหตุผลที่ล็อกเรียงตามรหัสสินค้า (บรรทัด 42): ถ้าสองใบล็อกสินค้าคนละลำดับ อาจรอกันเองไม่จบ (deadlock) การเรียงลำดับเดียวกันทำให้ไม่เกิด

UC-08 แสดงแดชบอร์ดปฏิบัติการ

ขั้นตอน	โปรแกรมทำอะไร	ไฟล์ : บรรทัด
1–2 เปิดหน้า คำนวณจากฐานข้อมูลทุกครั้ง	ไม่มีตารางสรุป ไม่มีแคช (BR-04)	backend/modules/dashboard/index.js : 20 · backend/app.js : 49
3 กล่องต้องดูแลวันนี้	คิวรอเกินเกณฑ์ · สินค้าต่ำกว่าจุดสั่งซื้อเพิ่ม · แจ้งเตือนที่ยังไม่อ่าน	backend/modules/dashboard/index.js : 31–39 · frontend/src/pages/dashboard-ops/index.jsx : 39, 45, 51
4 ยอดขายวันนี้ คิว รอจ่าย สินค้าที่ขาด	ผลรวมจาก orders และรายการจาก products	backend/modules/dashboard/index.js : 20–40
5 โหลดใหม่เมื่อมีเหตุการณ์ และทุก 60 วินาที	subscribe + setInterval	frontend/src/pages/dashboard-ops/index.jsx : 36

เปิดดูในเล่ม: Use Case X หัวข้อ 4.8 · บทที่ 3 รูปที่ 3.5 ตารางที่ 3.21 (แถวแดชบอร์ดปฏิบัติการ) รูปที่ 3.18 · บทที่ 2 หัวข้อ 2.1.8 (ทำไมคำนวณสด) · บทที่ 4 หัวข้อ 4.1.5 รูปที่ 4.7

ข้อมูล: อ่าน orders products notifications · AC: AC-09 · TC-08

ตัวอย่าง: หน้าจอบอกยอดวันนี้ 967 บาท 6 ใบ ใช้ SQL หาผลรวมตรงจากฐานข้อมูลก็ได้ 967 บาท 6 ใบเท่ากัน

UC-09 แสดงแดชบอร์ดระดับบริหาร

ขั้นตอน	โปรแกรมทำอะไร	ไฟล์ : บรรทัด
1 เปิดหน้า ค่าเริ่มต้น 30 วัน	เฉพาะผู้จัดการ	backend/app.js : 57 · backend/modules/dashboard/index.js : 50
2 กราฟแท่งรายวัน และสินค้าขายดี 10 อันดับ	รวมยอดตามวัน · เรียงตามยอดเงิน	backend/modules/dashboard/index.js : 55, 65–66
3 เลือก 7, 30 หรือ 90 วัน	ปุ่มเลือกช่วง	frontend/src/pages/dashboard-exec/index.jsx : 18

เปิดดูในเล่ม: Use Case X หัวข้อ 4.9 · บทที่ 3 รูปที่ 3.5 ตารางที่ 3.21 · บทที่ 4 หัวข้อ 4.1.6 รูปที่ 4.8

ข้อมูล: อ่าน orders order_items products · AC: ไม่มี (ดูหัวข้อ 13)

ตัวอย่าง: 30 วันล่าสุด เหล็ก 4 หุนขายดีสุด ผู้จัดการใช้ตัดสินใจสั่งเหล็ก 4 หุนเพิ่ม (EV-09)

UC-10 ค้นหาและดูประวัติการทำรายการ

ขั้นตอน	โปรแกรมทำอะไร	ไฟล์ : บรรทัด
(ทุกโมดูล) บันทึกเหตุการณ์	ฟังก์ชันกลางที่ทุกโมดูลเรียก	backend/modules/audit-log/index.js : 36
1 เปิดหน้า แสดงรายการของวันนี้	ค่าเริ่มต้นช่วงวันที่คือวันนี้	frontend/src/pages/audit-logs/index.jsx : 18–19
2 กรองช่วงวันที่และประเภทการกระทำ	เฉพาะผู้จัดการ	frontend/src/pages/audit-logs/index.jsx : 20 · backend/app.js : 59
3 เรียงใหม่ไปเก่า แสดงข้อมูลที่ถูกกระทำเป็นชื่อที่อ่านออก	เติมชื่อสินค้าหรือเลขที่ใบ	backend/modules/audit-log/index.js : 50, 71, 89, 118

เปิดดูในเล่ม: Use Case X หัวข้อ 4.10 · บทที่ 3 รูปที่ 3.5 · บทที่ 2 หัวข้อ 2.1.5 (บันทึกแบบเพิ่มอย่างเดียว) · บทที่ 4 หัวข้อ 4.1.9 รูปที่ 4.11

ข้อมูล: อ่าน audit_logs users และชื่อจาก orders order_items products · AC: AC-11 · TC-11

ตัวอย่าง: กรอง “ยืนยันจ่ายสินค้า” วันนี้ เห็นว่าใครจ่ายใบไหน เวลาเท่าไร จากไอพีไหน เหตุผลที่แสดงชื่อแทนรหัส (บรรทัด 89): ผู้จัดการอ่านรหัส #1532 แล้วไม่รู้ว่าเป็นอะไร

UC-11 รับการแจ้งเตือนภายในแอปพลิเคชัน

สิ่งที่เกิด	ระบบทำอะไร	ไฟล์ : บรรทัด
UC-07 หรือ UC-02 เปลี่ยนสต็อกสำเร็จ	ตรวจสินค้าที่ถูกแตะ ถ้าต่ำกว่าจุดสั่งซื้อเพิ่มสร้างแจ้งเตือน	backend/modules/alert/index.js : 42, 52 · เรียกจาก dispatch/index.js : 33 และ product/index.js : 171
มีใบเดิมที่ยังไม่อ่านอยู่แล้ว	ไม่สร้างซ้ำ	backend/modules/alert/index.js : 22–24
ตัวจับเวลาทุก 60 วินาที พบคิวรอเกิน 30 นาที	สร้างแจ้งเตือนคิวค้าง	backend/modules/alert/index.js : 63, 69, 73, 86
ผู้ใช้เปิดหน้าแจ้งเตือน	ที่ยังไม่อ่านขึ้นก่อน	backend/modules/alert/index.js : 92, 97 · backend/app.js : 50
ผู้จัดการกดอ่านแล้ว	เฉพาะผู้จัดการ บันทึกประวัติ	backend/modules/alert/index.js : 105 · backend/app.js : 58 · frontend/src/pages/notifications/index.jsx : 19
ตัวเลขบนเมนู	นับที่ยังไม่อ่าน ปรับเองเมื่อมีใบใหม่หรือกดอ่าน	frontend/src/services/unread.js : 19 · frontend/src/components/Layout.jsx : 44

เปิดดูในเล่ม: Use Case X หัวข้อ 4.11 · บทที่ 3 รูปที่ 3.5 ตารางที่ 3.5 (BR-05) · บทที่ 2 หัวข้อ 2.1.9 (ทำไมตรวจหลังสต็อกเปลี่ยน) · บทที่ 4 หัวข้อ 4.1.8 รูปที่ 4.10

ข้อมูล: อ่าน products orders · เขียน notifications audit_logs · AC: AC-10, AC-13 · TC-09, TC-10

ตัวอย่าง: จ่ายเหล็ก 2 หุนจนเหลือ 1,198 เส้น ต่ำกว่า 1,250 ระบบสร้างแจ้งเตือนเอง และเมนูแจ้งเตือนขึ้นเลข 1 ทันที โดยไม่มีใครเปิดหน้าดู (ภาพจริงในบทที่ 4 รูปที่ 4.7 และ 4.10)

10. เกณฑ์การยอมรับ (AC) ทุกข้อ

รูปแบบคือ เงื่อนไขตั้งต้น → การกระทำ → ผลที่ต้องเกิด ดู AC-01 ถึง AC-11 ในบทที่ 3 ตารางที่ 3.11 และ AC-12 ถึง AC-15 ใน Use Case X ภาคผนวก D คำพูดของอาจารย์ “เป็นตัวกำหนดเงื่อนไขว่าจะยอมรับได้ก็ต้องผ่านเงื่อนไขนี้” คือตารางนี้

AC	ถ้า...	เมื่อ...	ต้องได้...	ตัวอย่าง	ทำไมต้องมีข้อนี้	UC	TC	ผล
AC-01	มีบัญชีพนักงานขาย	เรียกเส้นทางของผู้จัดการตรง ๆ	เซิร์ฟเวอร์ปฏิเสธ	ได้ 401/403 ทั้งไม่มีโทเคน โทเคนปลอม โทเคนขาย	กันคนข้ามสิทธิ์	01	01	ผ่าน
AC-02	เข้าระบบเป็นผู้จัดการ	แก้จุดสั่งซื้อเพิ่ม	ค่าตรง และมีประวัติ	ตั้ง 7 ⟶ ฐานข้อมูล 7 และมีแถวแก้ไขสินค้า	จุดสั่งซื้อเพิ่มผิดจะเตือนผิด	02	02	ผ่าน
AC-03	มีสินค้าและสต็อกพอ	สร้างคำสั่งซื้อและยืนยันรับเงิน	ยอดถูก QR ยอดเดียวกัน ได้ใบรอจ่ายพร้อมเลขที่	525.00 บาท ทั้งตะกร้าและ QR	ยอดผิดคือเสียเงิน	03	03	ผ่าน
AC-04	มีคำสั่งซื้ออยู่แล้ว	สร้างต่อเนื่องและพร้อมกันหลายใบ	เลขที่ไม่ซ้ำ	120 ใบ ไม่ซ้ำ	เลขซ้ำคือตามของผิดใบ	04	04	ผ่าน
AC-05	บริการพิมพ์ดับ	กดยืนยันรับเงิน	ยังบันทึกและเข้าคิว พร้อมเตือน	ใบเข้าคิวปกติ จอขึ้นว่าพิมพ์ไม่สำเร็จ	เงินรับแล้ว จะเสียการขายเพราะเครื่องพิมพ์ไม่ได้	03	13	ผ่าน (ส่วนระบบ)
AC-06	เปิดหน้าคิวค้างไว้	หน้าร้านยืนยันรับเงินใบใหม่	ขึ้นหน้าคิวใน 3 วินาที ไม่กดรีเฟรช	สูงสุด 80 มิลลิวินาที	ตรง NFR-02	05	05	ผ่าน

AC	ถ้า…	เมื่อ…	ต้องได้…	ตัวอย่าง	ทำไมต้องมีข้อนี้	UC	TC	ผล
AC-07	มีใบรอจ่ายหนึ่งใบ	ส่งคำขอจ่ายพร้อมกันสองคำขอ	สำเร็จหนึ่ง อีกหนึ่งถูกปฏิเสธ สต็อกหักครั้งเดียว	ใบสั่ง 2 เส้น สต็อกลด 2 ไม่ใช่ 4	แก้ EV-06 โดยตรง	07	06	ผ่าน
AC-08	สต็อกน้อยกว่าที่สั่ง	ยืนยันจ่าย	ยกเลิกทั้งหมด ไม่มีอะไรเปลี่ยน	สั่ง 3 เหลือ 2 ⟶ ยังเหลือ 2 สถานะรอจ่าย	ห้ามจ่ายของที่ไม่มี	07	08	ผ่าน
AC-09	มีคำสั่งซื้อชำระแล้ว	เปิดแดชบอร์ดปฏิบัติการ	ตัวเลขตรงกับผลรวมจากฐานข้อมูล	967 บาท 6 ใบ ทั้งจอและ SQL	แดชบอร์ดผิดคือตัดสินใจผิด	08	09	ผ่าน
AC-10	ตั้งจุดสั่งซื้อเพิ่มไว้	จ่ายจนต่ำกว่าจุดนั้น	สร้างแจ้งเตือนเอง ไม่ซ้ำขณะยังไม่อ่าน	เหลือ 1,198 < 1,250 ได้ 1 ใบ	ตรง BR-05	11	10	ผ่าน
AC-11	มีการทำรายการแล้ว	ค้นตามช่วงเวลาและประเภท	แสดงผู้กระทำ เวลา ข้อมูลที่ถูกกระทำ ครบทุกแถว	กรองสร้างคำสั่งซื้อ ทุกแถวเป็นสร้างคำสั่งซื้อ	ตรวจย้อนหลังได้จริง	10	12	ผ่าน
AC-12	มีบัญชีที่เปิดใช้	เข้าระบบด้วยรหัสถูก	ได้โทเคน ไปหน้าแรกของบทบาท เห็นแต่เมนูตน · เข้าไม่สำเร็จได้ข้อความเดียวกันทุกกรณี	คลัง → /queue · ผิดชื่อหรือผิดรหัสขึ้นข้อความเดียวกัน	ข้อความต่างกันจะบอกคนร้ายว่าชื่อผู้ใช้ไหนมีจริง	01	01	ผ่าน
AC-13	มีใบค้างในคิว	รอเกินเกณฑ์โดยไม่มีใครเปิดจอ	สร้างแจ้งเตือนคิวค้าง ระบุเลขที่และเวลารอ	ใบรอ 45 นาที ได้แจ้งเตือน ใบใหม่ไม่ได้	เตือนเชิงรุก ไม่รอคนมาดู	11	11	ผ่าน
AC-14	เครื่องพิมพ์พร้อม	ยืนยันรับเงิน	พิมพ์เสร็จใน 5 วินาที	ยังไม่ได้วัดกับเครื่องจริง	ตรง NFR-05	03	13	ยังไม่ทวนสอบ
AC-15	10 เซสชันจากสามบทบาท	ใช้งานพร้อมกันต่อเนื่อง	ไม่มีคำขอล้มเหลว ผลถูกต้อง	15,383 คำขอ ใน 20 วินาที ล้มเหลว 0	ตรง NFR-06	ทุก UC	14	ผ่าน

11. ข้อมูล: UC ไหนใช้ตารางไหนใน ERD

คำพูดของอาจารย์ “มี data ที่ใช้ ซึ่งมันก็จะนำไปสู่ ERD” คือตารางนี้ ได้มาจากการไล่คำสั่งฐานข้อมูลในโค้ดของแต่ละโมดูล ดู ERD ในบทที่ 3 รูปที่ 3.9

ตาราง	เก็บอะไร	UC ที่อ่าน	UC ที่เขียน	เหตุผลการออกแบบ
users	ผู้ใช้ รหัสผ่านที่เข้ารหัส บทบาท	01, 05, 10	–	บทบาทมีแค่ 3 ค่า ตามตารางที่ 3.6
products	สินค้า หมวด ราคา หน่วย ยอดคงเหลือ จุดสั่งซื้อเพิ่ม	02, 03, 05, 07, 08, 09, 11	02, 07	ยอดคงเหลือห้ามติดลบ ฐานข้อมูลบังคับเองเป็นด่านสุดท้าย
orders	คำสั่งซื้อ เลขที่ ยอด สถานะ	04, 05, 06, 07, 08, 09, 11	03, 04, 06, 07	ไม่มีสถานะตะกร้า ใบเกิดเมื่อจ่ายเงินแล้วเท่านั้น ยอดขายจึงไม่เพี้ยน
order_items	รายการในแต่ละใบ พร้อมราคา ณ ตอนขาย	05, 07, 09, 10	03	เก็บราคาตอนขาย เพราะราคาวัสดุเปลี่ยนบ่อย ใบเก่าต้องไม่เปลี่ยนยอดตาม
stock_movements	ทุกครั้งที่สต็อกเปลี่ยน พร้อมยอดหลังเปลี่ยนและสาเหตุ	–	02, 07	เพิ่มอย่างเดียว ไม่แก้ไม่ลบ ย้อนดูได้ว่าเพี้ยนตั้งแต่ใบไหน
notifications	การแจ้งเตือนสินค้าใกล้หมดและคิวค้าง	08, 11	11	มีสถานะอ่านแล้ว ใช้กันการเตือนซ้ำ
audit_logs	ประวัติเหตุการณ์ ผู้กระทำ เวลา ไอพี	10	01, 02, 03, 06, 07, 11	ทุกโมดูลเขียน อ่านได้เฉพาะผู้จัดการ

ตรวจกลับ: ทุกตารางทั้ง 7 ตารางมี UC ใช้ ไม่มีตารางที่เกินมาโดยไม่มีใครใช้

12. ตารางสืบย้อนความต้องการฉบับเต็ม

อ่านจากซ้ายไปขวาคือจากปัญหาไปถึงผลทดสอบ อ่านจากขวาไปซ้ายคือจากผลทดสอบย้อนกลับไปหาว่าแก้ปัญหาอะไร ดูฉบับในเล่มที่บทที่ 3 ตารางที่ 3.12 และบทที่ 4 ตารางที่ 4.9

EV	FR / NFR / BR	US	UC	AC	TC	หน้าจอ	ตาราง	ผล
EV-01	FR-03, BR-01, NFR-05	US-01	UC-03, UC-04	AC-03, AC-04, AC-05, AC-14	TC-03, TC-04, TC-12	ขายหน้าร้าน	orders order_items	ผ่าน ยกเว้นพิมพ์กับเครื่องจริง
EV-02	FR-01, FR-09, NFR-01	US-01, US-06	UC-01, UC-10	AC-01, AC-11, AC-12	TC-01, TC-11	เข้าสู่ระบบ, ประวัติ	users audit_logs	ผ่าน
EV-03	FR-04, BR-03, NFR-02	US-02	UC-05	AC-06	TC-05	คิวรอจ่าย	orders	ผ่าน
EV-04	FR-04, BR-03	US-02	UC-05, UC-06	AC-06	TC-05	คิวรอจ่าย	orders	ผ่าน
EV-05	FR-05, NFR-03	US-03	UC-07	AC-07	TC-06	ยืนยันจ่าย	orders order_items	ผ่าน
EV-06	FR-06, BR-02, NFR-04	US-03	UC-07	AC-07	TC-06	ยืนยันจ่าย	orders products stock_movements	ผ่าน
EV-07	FR-06, BR-02	US-03	UC-07	AC-08	TC-07	ยืนยันจ่าย	products stock_movements	ผ่าน
EV-08	FR-07, BR-04	US-04	UC-08, UC-09	AC-09	TC-08	แดชบอร์ดสองระดับ	orders order_items products	ผ่าน
EV-09	FR-02, FR-08, BR-05	US-05	UC-02, UC-11	AC-02, AC-10, AC-13	TC-02, TC-09, TC-10	สินค้า, แจ้งเตือน	products notifications	ผ่าน

ตัวอย่างการอ่านหนึ่งแถว (EV-06): ร้านเคยโดนเอาบิลเก่ามารับของซ้ำ → จึงต้องมี FR-06 ตัดสต็อกในธุรกรรมเดียวและกันจ่ายซ้ำ ภายใต้กติกา BR-02 และต้องทนงานพร้อมกันตาม NFR-04 ⟶ พนักงานคลังพูดไว้ใน US-03 ⟶ ระบบทำใน UC-07 ขั้นที่ 4 ⟶ ยอมรับเมื่อผ่าน AC-07 ⟶ ทดสอบด้วย TC-06 ผ่าน → ผู้ใช้ทำที่หน้ายืนยันจ่ายและข้อมูลอยู่ในตาราง orders products stock_movements

13. เรื่องที่ยังไม่ครบ ต้องพูดตรง ๆ

เรื่อง	สถานะ	ถ้าอาจารย์ถาม ให้ตอบว่า
UC-06 และ UC-09 ไม่มี AC ของตัวเอง	ทดสอบระดับการรวมส่วนผ่านแล้ว แต่ไม่มีเกณฑ์การยอมรับกำกับ	ยอมรับว่าขาด และเสนอว่าจะเพิ่ม AC ให้สองกรณีนี้
TC-12 และ AC-14 พิมพ์กับเครื่องจริง	ผ่าน	พิมพ์ผ่าน USB บน Windows ภาษาไทยถูก ใช้เวลาประมาณ 2.3 วินาที วัดจากคลิปหนึ่งครั้ง
NFR-09 ติดตั้งจากคู่มือ	ยังไม่ทวนสอบ	ยังไม่ได้ให้คนนอกลองติดตั้ง
การประเมินความพึงพอใจ	ยังไม่ได้ทำ	รอให้ร้านใช้งานจริง

14. ซ้อมตอบคำถามอาจารย์

ลองตอบเองก่อนอ่านแนวคำตอบ แนวคำตอบเป็นแค่โครง ควรพูดด้วยคำของตัวเอง

คำถาม	แนวคำตอบ
ปัญหาคืออะไร	ร้านใช้บิลกระดาษ หน้าร้านกับคลังอยู่คนละที่ ลูกค้าต้องถือบิลเดินไปคลัง คลังอ่านลายมือผิด มีบิลเก่าถูกใช้ซ้ำ สต็อกไม่ตรง ผู้จัดการรู้ยอดตอนปิดร้าน (EV-01 ถึง EV-09)
เก็บความต้องการมาอย่างไร	ไปดูการทำงานจริงที่ร้านและถามเจ้าของร้านกับพนักงาน จดเป็นหลักฐานมีรหัส EV แล้วแปลงเป็น FR NFR BR
requirement มีอะไรบ้าง แบ่งอย่างไร	แบ่งสามกลุ่ม FR 9 ข้อคือสิ่งที่ระบบทำ NFR 9 ข้อคือทำได้ดีแค่ไหน BR 5 ข้อคือกติกาของร้าน
เอามาทำ use case ได้อะไร	11 กรณี ครอบคลุม FR ครบ 9 ข้อ แต่ละกรณีมีขั้นตอน AC หน้าจอ และตารางที่ใช้ (หัวข้อ 8–11)
UC ไหนสำคัญที่สุด เพราะอะไร	UC-07 เพราะแก้ปัญหาจ่ายซ้ำและสต็อกเพี้ยน ใช้การเปลี่ยนสถานะแบบมีเงื่อนไขในธุรกรรมเดียวกับการหักสต็อก
ถ้าสองคนกดจ่ายใบเดียวกันพร้อมกันจะเกิดอะไร	คนแรกเปลี่ยนสถานะได้ คนที่สองหาแถวที่ยังรอจ่ายไม่เจอ ธุรกรรมถูกยกเลิก ขึ้นข้อความว่าจ่ายไปแล้ว สต็อกหักครั้งเดียว (AC-07)
อะไรสัมพันธ์กับอะไร	พูดสายในหัวข้อ 1 แล้วยกตัวอย่าง EV-06 ในหัวข้อ 12
C2 ออกแบบอย่างไร	สี่ส่วนติดตั้ง: หน้าจอบนเบราว์เซอร์ ระบบหลังบ้าน บริการพิมพ์ (ต้องอยู่เครื่องที่ต่อเครื่องพิมพ์) และฐานข้อมูล · บทที่ 3 หัวข้อ 3.5.2
C3 ออกแบบอย่างไร	หลังบ้านแบ่ง 10 โมดูลตาม UC เช่น UC-07 ใช้ Dispatch Controller กับ Stock Deduction Service ที่แยกไว้เพราะเป็นตรรกะที่อ่อนไหวที่สุด · บทที่ 3 หัวข้อ 3.6
กล่องต้องดูแลวันนี้มาจากไหน เพิ่ม requirement ไหม	ไม่เพิ่ม เป็นการจัดข้อมูลของ FR-07 และ FR-08 ที่มีอยู่แล้วให้เห็นในที่เดียว เขียนไว้ใน UC-08 ขั้นที่ 3
อะไรยังไม่เสร็จ	ตอบตามหัวข้อ 13 ตรง ๆ อย่าบอกว่าผ่านในสิ่งที่ยังไม่ได้ทดสอบ